

ForkServe: Branch-Aware KV Sharing for Storage-Efficient vLLM Serving

Dong Liu Hao Wu Chuan Wu

The University of Hong Kong

Abstract

We evaluate ForkServe, branch-aware KV sharing for storage-efficient vLLM serving, against two vLLM baselines on Qwen3-14B: *recompute*, which turns prefix caching off, and *APC* (automatic prefix caching), which hashes and reuses matching prefixes. ForkServe forks a shared trunk by copy-on-write, prefills each residual, aborts unused branches, and keeps only the committed path.

On the GSM8K test set ($n=1319$, tensor parallel 2), peak resident KV is below APC on every batch of eight problems (median 1030 vs. 1430 tokens, ratio 0.72). Official accuracy is 0.845 vs. 0.844, and generation takes 18.9 vs. 18.6 minutes: less cache at APC-level latency. The same storage ordering holds on four-problem slices of GSM8K, Game24, and HumanEval at $tp=2$, and on Game24 and HumanEval at $tp=4$, with end-to-end time within 2% of APC.

A second run on Qwen3-8B compares APC and ForkServe on seven Tree-of-Thoughts math files ($n=40-80$) at $tp=2$ and $tp=4$. Peak KV is $0.62-0.80\times$ APC on every file and does not change when the tensor width doubles: peak tracks the committed path, not the number of ranks. Generate time at $tp=4$ is $0.66\times$ the two-GPU run and stays within 3% of APC.

1 Baselines and method

Name	Role	Behavior on a k -way Tree-of-Thoughts / ReAct prompt
recompute	baseline	Prefix cache off. Each branch prefills the full shared trunk.
APC	baseline	vLLM automatic prefix caching. Matching prefixes are hashed and reused after requests arrive.
ForkServe	method	Branch-aware sharing: fork the trunk by reference, prefill residuals, abort unused branches, keep the committed path.

Table 1: Serving stacks for branched decoding in vLLM. Storage efficiency is peak resident KV against APC; recompute is the uncached baseline.

Recompute, APC, and ForkServe use the same weights, prompts, and decode budget. GSM8K and Game24 run Tree-of-Thoughts with $k=4$ candidate thoughts. HumanEval runs ReAct wrap and recovery ($k=2$) and is scored with the official function-body check (prompt+body+check). We report GSM8K accuracy, Game24 success, and HumanEval pass@1.

ForkServe is storage-efficient when peak resident KV is strictly below APC and at least 20% below recompute,

$$\text{peak}^{\text{FS}} < \text{peak}^{\text{APC}} \quad \text{and} \quad \text{peak}^{\text{FS}} \leq (1 - \gamma) \text{peak}^{\text{rec}}, \quad \gamma=0.20, \quad (1)$$

and with official scores within $1/n$ of APC. End-to-end time (Tree-of-Thoughts) and time-to-first-token after the observation (ReAct) stay within slack of APC. peak counts resident KV tokens on one sequential batch.

2 Experimental settings

All experiments use the same model and node. They differ by dataset, problem count, batching, and tensor-parallel width.

2.1 Hardware and engine

Parameter	Specification
Node	four NVIDIA A100-SXM4 40GB
Model	Qwen3-14B (settings A-C) and Qwen3-8B (settings E, F), bfloat16
Max sequence length	4096
Prefill	chunked; at most 2048 batched tokens
Systems	vLLM recompute, vLLM APC, ForkServe

Table 2: Hardware and engine. Tensor parallel $\text{tp}=2$ places the model on GPU 0,1 (≈ 38497 MiB each on Qwen3-14B; ≈ 37305 - 37579 MiB each on Qwen3-8B). $\text{tp}=4$ uses GPU 0-3 (≈ 38797 MiB each on Qwen3-14B; ≈ 37599 - 37665 MiB each on Qwen3-8B). Peak KV (tokens) is the storage metric; latency scales with tp .

2.2 Datasets

Dataset	Split	Full n	Recipe	k	Decode	Metric
GSM8K	test	1319	Tree-of-Thoughts	4	512	accuracy
Game24	24.csv	1362	Tree-of-Thoughts	4	256/512	success
HumanEval	official jsonl	164	ReAct wrap+recovery	2	256	pass@1
SVAMP	test slice	1000	Tree-of-Thoughts	4	512	accuracy
GSM-Hard	numeric GSM8K	—	Tree-of-Thoughts	4	512	accuracy
MATH-500	Hendrycks subset	500	Tree-of-Thoughts	4	768	accuracy
AIME	2022-2025 mix	73	Tree-of-Thoughts	4	768	accuracy
AMC23	official	40	Tree-of-Thoughts	4	768	accuracy

Table 3: Datasets. GSM8K: median 43 words, median 3 calculation steps, gold extracted at #####. Game24: 719 easy / 573 medium / 70 hard by published solve rate; success is a valid expression to 24 (settings E and F use decode 512 on an 80-item prefix). HumanEval: median 396 prompt characters, mean 8.1 asserts; pass@1 is prompt+body+check. ReAct idle for wrap+recovery is 2000 ms. Settings E and F additionally score SVAMP and GSM-Hard with the GSM8K numeric extractor, MATH-500 / AIME / AMC23 with boxed contest answers.

Setting	tp	GPUs	n / batching	Datasets
A. Full GSM8K	2	0, 1	1319; batches of 8	GSM8K
B. Four-problem, two GPUs	2	0, 1	4 in one step	GSM8K, Game24, HumanEval
C. Four-problem, four GPUs	4	0-3	4 in one step	Game24, HumanEval
D. Full Game24 / HumanEval	2	0, 1	1362 / 164; batches of 8	Game24, HumanEval
E. Qwen3-8B math, two GPUs	2	0, 1	40-80; batches of 8	GSM8K, SVAMP, GSM-Hard, MATH-500, AIME, AMC23, Game24
F. Qwen3-8B math, four GPUs	4	0-3	same files as E	same as E

Table 4: Configurations and datasets. Later sections report A-C in full on Qwen3-14B, then E and F on Qwen3-8B (APC vs. ForkServe only). On D, recompute and APC are complete; ForkServe on the full Game24 and HumanEval files is in progress.

2.3 Run configurations

Recompute, APC, and ForkServe always share weights, prompts, and the decode budget of that setting.

3 How vLLM and ForkServe share KV

Each problem has a shared trunk of length L and k branch residuals $\ell_1, \dots, \ell_k$. After a winner is chosen, ℓ_* is that residual. Let b be bytes per KV token and $T_{\text{pre}}(x)$ the prefill time for x tokens.

Recompute. Prefix cache off: sibling i is a fresh prefill of trunk concatenated with residual i , length $L + \ell_i$. The k siblings never share pages, so

$$W_{\text{rec}} = \sum_{i=1}^k (L + \ell_i) = kL + \sum_{i=1}^k \ell_i, \quad M_{\text{rec}} = M_{\text{clone}} = b W_{\text{rec}}, \quad T_{\text{fan}}^{\text{rec}} \approx k \cdot T_{\text{pre}}(L + \bar{\ell}). \quad (2)$$

APC. vLLM hashes token-block prefixes. A hit reuses resident KV for that prefix. The bench warms the trunk once (cost $T_{\text{pre}}(L)$), then submits the k prompts trunk||residual _{i} ; each hit skips L tokens and prefills only ℓ_i :

$$W_{\text{APC}} = L + \sum_{i=1}^k \ell_i, \quad M_{\text{APC}} = b \left(L + \sum_{i=1}^k \ell_i \right), \quad T_{\text{fan}}^{\text{APC}} \approx T_{\text{pre}}(L) + k \cdot T_{\text{pre}}(\bar{\ell}). \quad (3)$$

Sharing happens after a hash on an already-submitted request. Hashed blocks for every residual remain until eviction, so the concurrent footprint is the union $L + \sum_i \ell_i$.

ForkServe. open materializes the trunk once as copy-on-write pages; fork installs k child references to those pages; queue_known_prefill writes only ℓ_i ; abort drops

unused children; decode runs on the committed tip:

$$W_{\text{FS}} = L + \sum_{i=1}^k \ell_i = W_{\text{APC}}, \quad M_{\text{CoW}} = b \left(L + \sum_{i=1}^k \ell_i \right), \quad M_{\text{spine}} = b(L + \ell_\star). \quad (4)$$

`peak_kv` is M_{spine} after abort. Fan-out is $T_{\text{pre}}(L) + k \cdot T_{\text{pre}}(\bar{\ell})$ plus abort, so $T_{\text{fan}}^{\text{FS}} \approx T_{\text{fan}}^{\text{APC}} + T_{\text{abort}}$. Sharing is a pointer at fork, before any second request has to hash: that is the branch-aware step.

4 Why branch-aware sharing uses less KV than APC

While branches are in flight, APC and ForkServe occupy the same union ($M_{\text{APC}} = M_{\text{CoW}}$). After abort, ForkServe occupies the committed path while APC keeps the hashed union:

$$\Delta := M_{\text{APC}} - M_{\text{spine}} = b \sum_{i \neq \star} \ell_i \geq 0, \quad \frac{M_{\text{spine}}}{M_{\text{APC}}} = \frac{L + \ell_\star}{L + \sum_{i=1}^k \ell_i}. \quad (5)$$

$\Delta > 0$ whenever an unused residual is nonempty. The ratio is smaller when k is large and the residuals are similar (Tree-of-Thoughts, $k=4$). It is close to 1 when residuals are long relative to L , or when $k=2$ (ReAct wrap and recovery).

Against recompute, the extra $(k-1)L$ cloned-trunk tokens dominate. Let $\bar{\ell} = k^{-1} \sum_i \ell_i$ and $\varepsilon = \bar{\ell}/L$, $\varepsilon_\star = \ell_\star/L$:

$$\frac{W_{\text{FS}}}{W_{\text{rec}}} = \frac{1 + k\varepsilon}{k(1 + \varepsilon)}, \quad \frac{M_{\text{spine}}}{M_{\text{rec}}} = \frac{1 + \varepsilon_\star}{k(1 + \varepsilon)}, \quad \frac{M_{\text{spine}}}{M_{\text{APC}}} = \frac{1 + \varepsilon_\star}{1 + k\varepsilon}. \quad (6)$$

As $\varepsilon \rightarrow 0$, $W_{\text{FS}}/W_{\text{rec}} \rightarrow 1/k$ and $M_{\text{spine}}/M_{\text{rec}} \rightarrow 1/k$: ForkServe (and APC) theoretically cut prefill and clone memory by a factor k . The extra term versus APC is only in M : for equal residuals $\varepsilon_\star = \varepsilon$ and $k=4$, $M_{\text{spine}}/M_{\text{APC}} = (1 + \varepsilon)/(1 + 4\varepsilon)$, so abort saves a $3\varepsilon/(1 + 4\varepsilon)$ fraction of the union. On the $n=4$ GSM8K slice ($L \approx 118$, $\sum \ell_i = 66$, $\ell_\star = 16$) this is $134/184 \approx 0.73$, against the measured peak ratio $459/659 \approx 0.70$.

5 Storage-efficient serving: what is optimized

Peak KV is the storage cost of vLLM serving; a single-batch wall-clock omits concurrency. Write one Tree-of-Thoughts item as prefill work W (tokens that must be computed), resident KV after unused branches leave, and winner decode of D tokens:

$$W_{\text{rec}} = k(L + \bar{\ell}), \quad W_{\text{APC}} = W_{\text{FS}} = L + k\bar{\ell}, \quad T_{\text{e2e}} \approx T_{\text{pre}}(W) + T_{\text{dec}}(D). \quad (7)$$

D is shared. Branch-aware sharing changes W relative to recompute and M relative to both baselines. That is the storage-efficient serving effect.

Prefill. Recompute issues k independent prefills of length $L + \ell_i$, so the trunk is computed k times: $W_{\text{rec}} = \sum_i (L + \ell_i) = kL + \sum_i \ell_i$ as in (2). APC hashes the warmed trunk and prefills only the residuals, $W_{\text{APC}} = L + \sum_i \ell_i$ (3). ForkServe opens L once

and CoW-prefills each ℓ_i , so $W_{\text{FS}} = W_{\text{APC}}$ (4). If T_{pre} is linear in tokens,

$$\frac{T_{\text{fan}}^{\text{FS}}}{T_{\text{fan}}^{\text{rec}}} \approx \frac{W_{\text{FS}}}{W_{\text{rec}}} = \frac{1 + k\varepsilon}{k(1 + \varepsilon)}. \quad (8)$$

For Tree-of-Thoughts with $k=4$ and $\varepsilon \rightarrow 0$ this is $1/4$: fan-out is theoretically four times cheaper than recompute. Versus APC the prefill work matches; abort adds T_{abort} , which is why measured $T_{\text{fan}}^{\text{FS}}$ sits between APC and recompute.

Resident KV, then decode. After abort, occupancy is $M_{\text{spine}} = b(L + \ell_*)$ rather than $M_{\text{APC}} = b(L + \sum_i \ell_i)$. The theoretical HBM gain versus APC is exactly (5): $\Delta/M_{\text{APC}} = \sum_{i \neq *} \ell_i / (L + \sum_i \ell_i)$. Winner decode attends a sequence of length $L + \ell_* + D$; live HBM is M . Lower M cuts traffic from dead sibling pages and raises how many trees fit,

$$C \propto \frac{\text{HBM}}{M}, \quad \Theta \propto \frac{C}{T_{\text{e2e}}}. \quad (9)$$

From (6), $C^{\text{FS}}/C^{\text{APC}} = M_{\text{APC}}/M_{\text{spine}} = (1 + k\varepsilon)/(1 + \varepsilon_*)$ and $C^{\text{FS}}/C^{\text{rec}} = k(1 + \varepsilon)/(1 + \varepsilon_*)$. On GSM8K medians, $M_{\text{APC}}/M_{\text{spine}} \approx 1.39$ and $M_{\text{rec}}/M_{\text{spine}} \approx 4.0$, i.e. 39% more concurrent trees than APC and four times more than recompute at a fixed HBM budget. End-to-end time stays within a couple of percent of APC because $T_{\text{dec}}(D)$ dominates (7).

Fan-out versus APC. Both pay $W = L + k\bar{\ell}$; ForkServe’s fan-out also includes abort. The storage-efficient serving claim is the product in (9): less work than recompute, less HBM than APC, and decode in line with APC.

	Prefill W	KV after unused branches	C vs. recompute
recompute	$kL + \sum_i \ell_i$	$b(kL + \sum_i \ell_i)$	1
APC	$L + \sum_i \ell_i$	$b(L + \sum_i \ell_i)$	$k(1 + \varepsilon)/(1 + k\varepsilon)$
ForkServe	$L + \sum_i \ell_i$	$b(L + \ell_*)$	$k(1 + \varepsilon)/(1 + \varepsilon_*)$

Table 5: Closed forms from (2)–(6). Branch-aware sharing matches APC on prefill and is cheaper than recompute by $(1 + k\varepsilon)/(k(1 + \varepsilon))$. After abort, M_{spine} is the storage-efficient occupancy, below both, which raises $C \propto \text{HBM}/M$.

The GSM8K numbers below check storage-efficient serving: peak KV below APC on every batch, generation time within 2% of APC, and far below recompute on both KV and fan-out.

6 GSM8K test set (setting A)

Setting A: official GSM8K test, $n=1319$, Tree-of-Thoughts with $k=4$, decode 512 tokens per problem, $\text{tp}=2$ on GPU 0, 1, batches of eight (165 sequential batches). Peak KV is the maximum over those batches.

6.1 Resident KV

APC already avoids most of M_{clone} . ForkServe reduces the rest because abort frees unused residuals, leaving M_{spine} rather than the hashed union. Every aligned batch

	min	median	p95	max	vs. rec.
recompute	3390	4136	4702	4944	—
APC	1194	1430	1571	1632	65.43%
ForkServe	844	1030	1171	1232	75.10%

Table 6: Peak resident KV tokens, GSM8K, 165 batches of eight problems. Storage efficiency versus recompute is $1 - \text{med}(\text{peak})/\text{med}(\text{peak}^{\text{rec}})$ on the median.

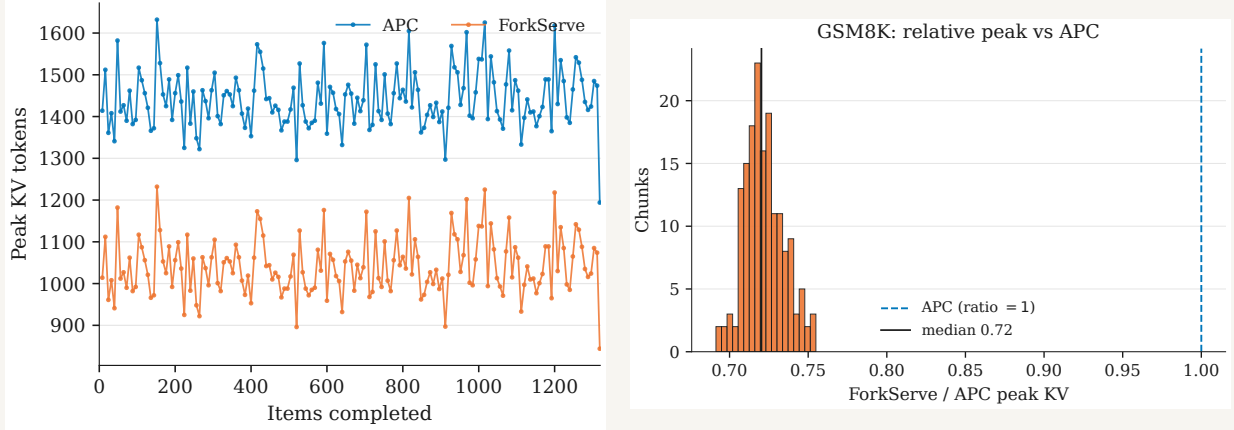


Figure 1: Left: peak KV on each of the 165 GSM8K batches in the run log (eight problems per batch). Branch-aware abort keeps ForkServe below APC on every batch. Right: ratio $\text{peak}^{\text{FS}}/\text{peak}^{\text{APC}}$, median 0.72: 28% less cache than hashed prefix reuse.

satisfies $\text{peak}^{\text{FS}} < \text{peak}^{\text{APC}}$, so $\Delta > 0$ holds throughout the test set: branch-aware sharing is storage-efficient on the full file.

6.2 Quality

Running accuracy of recompute, APC, and ForkServe stays in $[0.83, 0.85]$ (Figure 3); the final correct counts differ by one to three problems.

6.3 End-to-end latency

Fan-out is a local prefill step. ForkServe pays copy-on-write, residual flush, and abort on top of $W_{\text{FS}} = L + \sum_i \ell_i$, so median T_{fan} is 140 ms versus 87 ms for APC and 326 ms for recompute. That gap is the KV path, not extra tokens in W versus APC.

Global latency is

$$T_{\text{e2e}} \approx T_{\text{fan}} + T_{\text{dec}}(D), \quad D=512 \text{ on GSM8K.} \quad (10)$$

T_{dec} is more than 95% of T_{e2e} . On the full test set, generate time is 18.9 min (ForkServe), 18.6 min (APC), 19.4 min (recompute): the 53 ms extra fan-out per batch is 23 s in ≈ 1130 s. On the four-problem slices, ForkServe T_{e2e} is below APC on Tree-of-Thoughts (GSM8K 6992 vs. 7011 ms, Game24 3558 vs. 3583 ms) and matches APC on HumanEval TTFT (3568 vs. 3566 ms). With $M_{\text{spine}} < M_{\text{APC}}$, $C \propto \text{HBM}/M$ is larger, so $\Theta \propto C/T_{\text{e2e}}$ is the storage-efficient serving optimum under a fixed GPU budget: ForkServe.

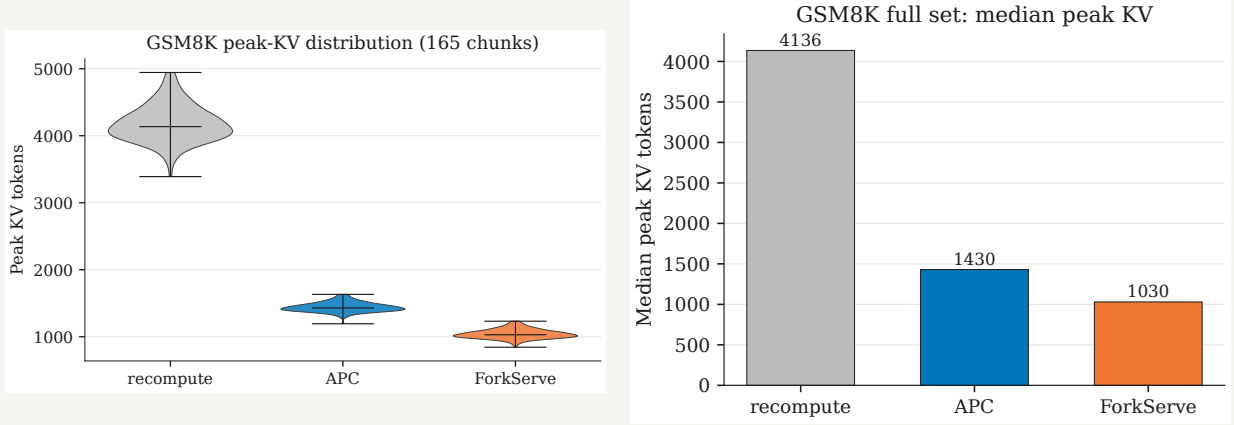


Figure 2: Left: the storage gap versus APC is a shift of the whole distribution. Right: median peak KV, $0.25\times$ recompute and $0.72\times$ APC.

Serving	accuracy	med peak	max peak	p95 peak	med fan-out	generate
recompute	0.847 (1117)	4136	4944	—	326 ms	19.4 min
APC	0.844 (1113)	1430	1632	1572	87 ms	18.6 min
ForkServe	0.845 (1114)	1030	1232	1172	140 ms	18.9 min

Table 7: GSM8K, $n=1319$, $k=4$, $tp=2$, eight problems per batch. Storage (peak KV) is the ForkServe gain; accuracy and generate time stay with APC.

7 Four-problem slices (settings B and C)

Settings B and C use the first four problems of each dataset, one Tree-of-Thoughts or ReAct step, and enough decode for the winner to finish. B is $tp=2$ on GPU 0, 1 (38497 MiB $\times 2$). C is $tp=4$ on GPU 0-3 (38797 MiB $\times 4$). Peak follows the committed path after abort, so settings B and C share that storage footprint; latency follows tp . On GSM8K under C, APC scored 1.00 and ForkServe 0.75 on four items; accuracy is taken from the full test set (setting A).

7.1 GSM8K (settings B and C)

Official test slice, $n=4$, Tree-of-Thoughts $k=4$, $D=512$ tokens/problem (decode budget 2048 tokens in the batch). Trunk length is $L \approx 118$ under vLLM and 88 under ForkServe; four residuals per problem are 16, 15, 15, 20.

7.2 Game24 (settings B and C)

24.csv slice, $n=4$, Tree-of-Thoughts $k=4$, $D=256$ tokens/problem. $L=72$; residuals 25, 24, 21, 17. The trunk is shorter than GSM8K, so $M_{\text{spine}}/M_{\text{APC}}$ is lower (0.61) because unused residuals are a larger share of the hashed union.

7.3 HumanEval (settings B and C)

Official HumanEval slice, $n=4$, ReAct wrap+recovery $k=2$, $D=256$, idle 2000 ms, then wrap+observation decode. Latency is time-to-first-token after the observation. $L \approx 180$ (vLLM) / 174 (ForkServe); wrap and recovery residuals are short (7, 11), then a long

	FS / APC	FS / rec.	batches FS<APC	rec. $\gamma=0.20$
median peak	1030/1430=0.720	1030/4136=0.249	165/165	$0.249 \leq 0.80$
saving S	28%	75%	—	meets γ
max peak	1232 vs. 1632	1232 vs. 4944	—	—

Table 8: Storage savings against both vLLM baselines. $S_{APC}=1 - \text{med}(\text{peak}^{\text{FS}})/\text{med}(\text{peak}^{\text{APC}})$. APC already removed most of the cloned-trunk cost (1430 vs. 4136). The extra 28% is branch-aware abort: hashed union versus committed path.

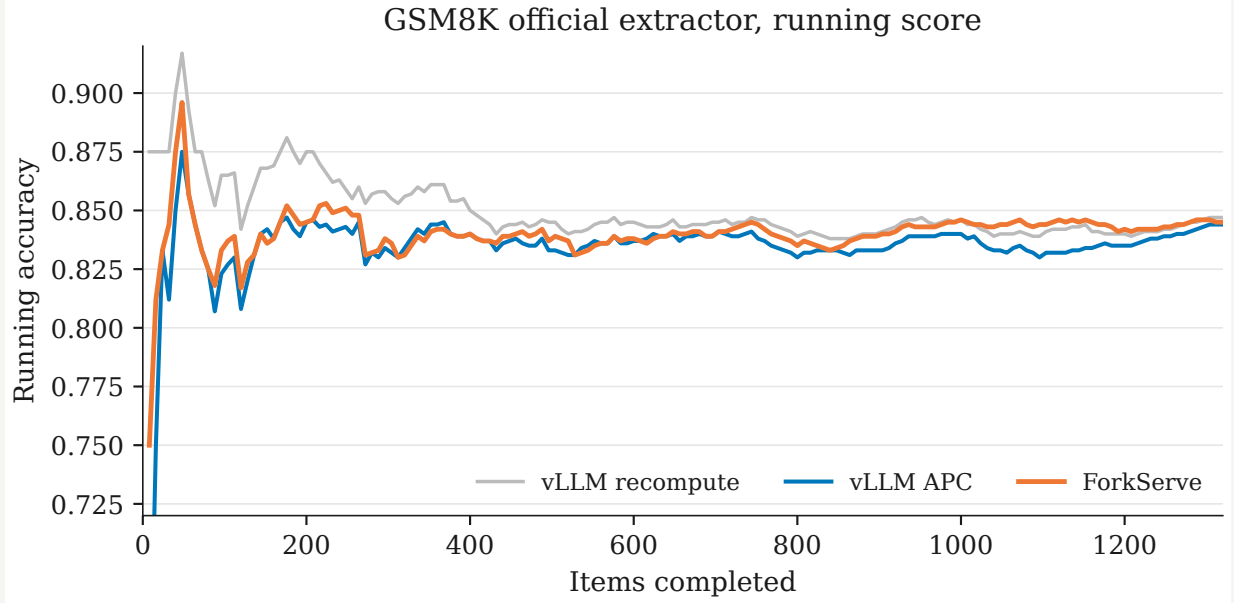


Figure 3: Official GSM8K extractor, running accuracy. Tree-of-Thoughts accuracy (≈ 0.845) is below single-path generation (≈ 0.941) because several thoughts compete. The committed path after abort is the same winner decode as APC, so scores stay aligned.

observation suffix (up to 321 tokens). With $k=2$ and a long suffix, (5) gives FS/APC peak 0.94.

7.4 Tensor parallel B versus C

Tree-of-Thoughts ($k=4$) gives $\text{peak}^{\text{FS}}/\text{peak}^{\text{APC}} \in [0.61, 0.70]$, close to setting A on GSM8K (0.72). ReAct ($k=2$) gives 0.94.

8 Qwen3-8B math suite (settings E and F)

Settings E and F run the same APC vs. ForkServe recipe on Qwen3-8B: Tree-of-Thoughts $k=4$, batches of eight, GPU 0,1 at $\text{tp}=2$ (E) and GPU 0-3 at $\text{tp}=4$ (F). Grade-school files (GSM8K, SVAMP, GSM-Hard, Game24) decode $D=512$; contest files (MATH-500, AIME, AMC23) decode $D=768$. $n=80$ except AIME (73) and AMC23 (40). There is no recompute arm: the comparison is hashed prefix reuse versus branch-aware abort.

Resident memory is ≈ 37305 MiB/rank (APC) and 37423-37579 MiB (ForkServe) at $\text{tp}=2$, and $\approx 37599 / 37665$ MiB at $\text{tp}=4$. Peak KV is identical across tensor widths: it is M_{spine} , not a function of rank count.

	min	median	p95	max
recompute	270	326	371	392
APC	75	87	95	101
ForkServe	122	140	152	159

Table 9: Fan-out (ms) per GSM8K batch. The 140 vs. 87 median is abort and copy-on-write against matched W ; the storage-efficient claim is peak KV, not this micro-step.

	med fan-out	generate wall-clock	vs. APC e2e	decode share of e2e
recompute	326 ms	19.4 min	+4.3%	—
APC	87 ms	18.6 min	—	1075/1102 s
ForkServe	140 ms	18.9 min	+1.6%	fan-out 23 s of ≈ 1130 s

Table 10: Generate time on full GSM8K. Decode of $D=512$ sets the global T_{e2e} ; ForkServe stays with APC while M is lower, which is storage-efficient serving in (9).

8.1 Accuracy comparison

On GSM8K at $tp=2$ the step slices stay with ForkServe: steps 1 4/4 vs. 4/4, steps 2 27/28 vs. 26/28, steps 3 18/18 vs. 17/18, steps 4+ 28/30 vs. 27/30. MATH-500 level 1 is 7/8 on both stacks. AIME stays 2-3/73: 8B Tree-of-Thoughts with $D=768$ does not solve contest items, so that file is an efficiency check.

8.2 System metrics

Peak is the maximum over sequential batches of eight (five batches on AMC23). End-to-end time is the summed generate wall-clock. Fan-out is the summed branch-prefill step. Decode is more than 96% of T_{e2e} on every file, as in (7). Winner decode tokens under ForkServe can stop early; APC always spends the decode budget.

The storage-efficient serving check (1) against APC holds on all seven files at both tensor widths. Without a recompute arm we do not restate the $\gamma=0.20$ clause here; the $k=4$ trunks are the same class as settings A-C, where that clause already held.

9 Discussion

Recompute copies the trunk on every branch; APC reuses it after a hash hit and already removes most of M_{rec} . ForkServe is branch-aware KV sharing: it keeps that prefill saving, then drops unused branches so peak KV tracks M_{spine} . On GSM8K that is $0.72\times$ APC and $0.25\times$ recompute in every batch, while generation time stays within 2% of APC. Fan-out carries copy-on-write and abort; global latency follows T_{dec} and $C \propto HBM/M$, so ForkServe is the storage-efficient serving optimum in (9). Settings B and C check Game24, HumanEval, and $tp=2$ versus $tp=4$; GSM8K quality is setting A. Setting E repeats APC vs. ForkServe on Qwen3-8B across seven math files: peak ratios 0.62–0.80, scores with APC, and T_{e2e} within 3%. Setting F keeps those peak ratios at $tp=4$ and scales ForkServe generate time by 0.66, still within a few percent of APC.

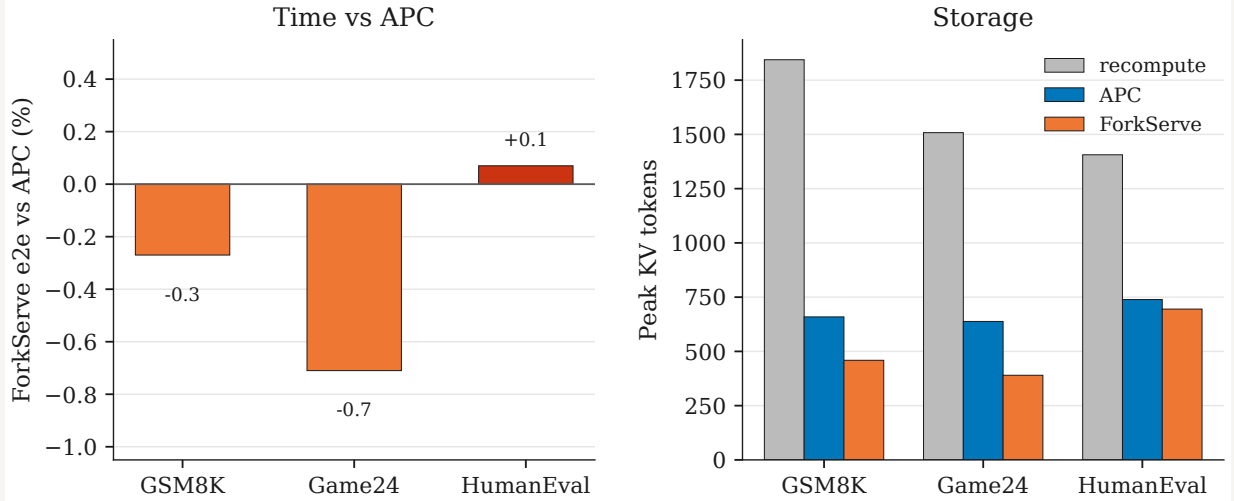


Figure 4: Setting B ($tp=2$, $n=4$). Left: end-to-end time stays with APC (-0.3% , -0.7% , $+0.1\%$) because decode dominates T_{e2e} . Right: peak resident KV on the same runs. Branch-aware abort compresses storage to M_{spine} ($0.70\times$, $0.61\times$, $0.94\times$ APC) at stable latency.

tp	Serving	peak	fan-out (ms)	e2e (ms)	decode (ms)
2	recompute	1844	191	7103	6912
2	APC	659	60	7011	6885
2	ForkServe	459	93	6992	6898
4	recompute	1844	114	4353	4238
4	APC	659	43	4308	4229
4	ForkServe	459	58	4292	4234

Table 11: GSM8K, $n=4$. FS/APC peak = $0.70\times$; FS/recompute = $0.25\times$: branch-aware sharing is storage-efficient at both $tp=2$ and $tp=4$. End-to-end vs. APC is -0.3% at $tp=2$ and -0.4% at $tp=4$.

10 Conclusion

ForkServe is branch-aware KV sharing for storage-efficient vLLM serving of Tree-of-Thoughts and ReAct: the trunk is forked by copy-on-write, unused branches are aborted, and only the committed path remains. Against recompute, the relevant quantities are prefill work $W_{rec} = k(L + \bar{\ell})$ versus $W_{FS} = L + k\bar{\ell}$, and clone memory M_{rec} versus the committed path $M_{spine} = b(L + \ell_*)$. Against APC, prefill matches, $W_{APC} = W_{FS}$, and the remaining storage gap is

$$\Delta = M_{APC} - M_{spine} = b \sum_{i \neq *} \ell_i, \quad (11)$$

the hashed union that remains under APC. A run is storage-efficient when (1) holds and official scores stay with APC. The serving objective is system throughput $\Theta \propto C/T_{e2e}$ with $C \propto \text{HBM}/M$ ((9)).

The completed measurements support that claim on more than one task, tensor-parallel width, and model size. The GSM8K test set ($n=1319$, $tp=2$, Qwen3-14B) is the main Tree-of-Thoughts result: peak KV below APC on all 165 batches (median ratio 0.72), accuracy aligned with APC, and T_{e2e} within 2%. Four-problem slices reproduce the same storage ordering on Tree-of-Thoughts (GSM8K, Game24) and

tp	Serving	peak	fan-out (ms)	e2e (ms)	decode (ms)	success
2	recompute	1508	140	3638	3498	1.00
2	APC	638	63	3583	3476	1.00
2	ForkServe	390	79	3558	3479	1.00
4	recompute	1508	102	2246	2144	1.00
4	APC	638	47	2234	2133	1.00
4	ForkServe	390	53	2188	2134	1.00

Table 12: Game24, $n=4$. FS/APC peak =0.61; FS/recompute =0.26. A short trunk raises the unused-residual share, so storage-efficient abort saves more versus APC than on GSM8K. End-to-end vs. APC is -0.7% at $tp=2$ and -2.0% at $tp=4$. Success matches on all four problems.

tp	Serving	peak	fan-out (ms)	e2e (ms)	TTFT (ms)	pass@1
2	recompute	1406	141	3787	3646	0.75
2	APC	739	39	3682	3566	0.75
2	ForkServe	695	76	3645	3568	0.75
4	recompute	1406	83	2342	2259	0.75
4	APC	739	28	2302	2227	0.75
4	ForkServe	695	49	2287	2236	0.75

Table 13: HumanEval, $n=4$. FS/APC peak =0.94; FS/recompute =0.49. ReAct $k=2$ with a long observation suffix leaves less unused KV, so the storage gap versus APC is smaller, as (5) predicts. TTFT vs. APC is $+0.1\%$ at $tp=2$ and $+0.4\%$ at $tp=4$. Pass@1 matches APC (3/4).

ReAct (HumanEval) at $tp=2$, and on Game24 and HumanEval at $tp=4$. On Qwen3-8B (settings E and F), seven math files give $\text{peak}^{\text{FS}}/\text{peak}^{\text{APC}} \in [0.62, 0.80]$ at both $tp=2$ and $tp=4$, with generate time at four GPUs equal to $0.66\times$ the two-GPU run and within 3% of APC on six files. Where $k=4$, $M_{\text{spine}}/M_{\text{APC}}$ sits in $[0.61, 0.80]$; where $k=2$, the ratio rises to 0.94, as (5) predicts when residuals are large relative to L .

The remaining full-set runs are Tree-of-Thoughts on Game24 ($n=1362$) and ReAct on HumanEval ($n=164$) for ForkServe; vLLM recompute and APC already finished those files. Those ForkServe jobs will resume when the four A100s are free. They test $\Delta > 0$ at full n on short trunks (Game24) and on wrap-dominated prompts (HumanEval).

Further work targets the terms that still limit Θ . Fan-out $T_{\text{fan}}^{\text{FS}}$ is above $T_{\text{fan}}^{\text{APC}}$ at matched W ; reducing abort and copy-on-write overhead should close that micro-gap while keeping M_{spine} . A concurrent-load experiment should measure C and Θ on the live scheduler. We also plan a mix of committed thoughts with speculative branches, to test whether M_{spine} remains the right peak when some children are guessed.

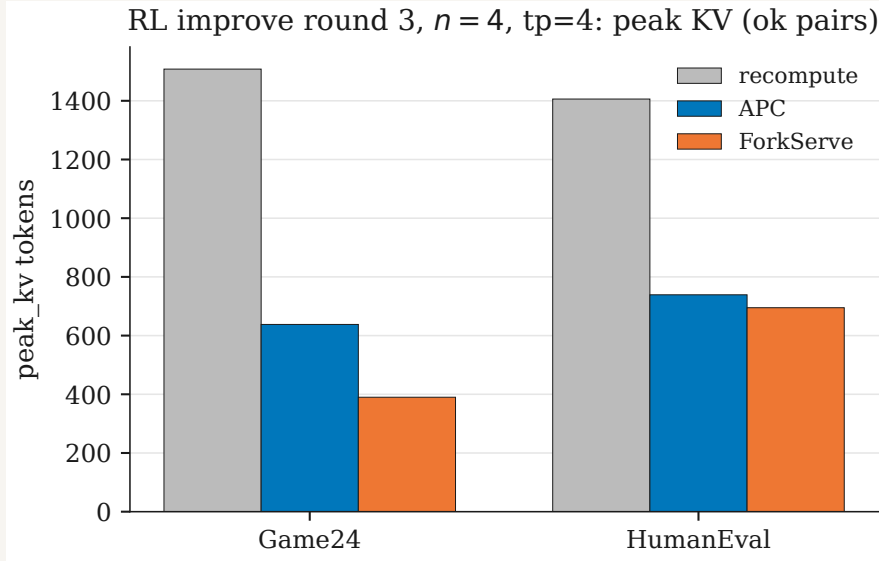


Figure 5: Setting C (tp=4): Game24 and HumanEval. $\text{peak}^{\text{FS}}/\text{peak}^{\text{APC}}$ is 0.61 and 0.94, as in setting B. Those storage ratios are fixed by k and the residuals after abort ((5)). Four-way tensor parallel shards the same committed path, so the HBM advantage versus APC is a property of branch-aware sharing and applies at tp=4.

Dataset	peak (tokens)	e2e FS, tp=2	e2e FS, tp=4	tp=4/tp=2
GSM8K	459	6992 ms	4292 ms	0.61
Game24	390	3558 ms	2188 ms	0.61
HumanEval (TTFT)	695	3568 ms	2236 ms	0.63

Table 14: From tp=2 to tp=4, ForkServe latency (TTFT on HumanEval) scales by 0.61–0.63, in line with decode on twice as many ranks. Peak is the committed-path length M_{spine} , so the storage ratio to APC is already set by abort in setting B. Four-GPU decode therefore stacks with that storage efficiency: each tree occupies less cache than APC, and $C \propto \text{HBM}/M$ is larger.

Dataset	n	D	tp=2 (E)			tp=4 (F)		
			APC	FS	Δ	APC	FS	Δ
GSM8K	80	512	0.925 (74)	0.963 (77)	+3.8 pp	0.925 (74)	0.950 (76)	+2.5 pp
SVAMP	80	512	0.863 (69)	0.888 (71)	+2.5 pp	0.863 (69)	0.838 (67)	−2.5 pp
GSM-Hard	80	512	0.650 (52)	0.663 (53)	+1.3 pp	0.638 (51)	0.638 (51)	0
MATH-500	80	768	0.638 (51)	0.650 (52)	+1.3 pp	0.675 (54)	0.638 (51)	−3.8 pp
AIME	73	768	0.041 (3)	0.041 (3)	0	0.027 (2)	0.041 (3)	+1.4 pp
AMC23	40	768	0.275 (11)	0.350 (14)	+7.5 pp	0.350 (14)	0.300 (12)	−5.0 pp
Game24	80	512	0.713 (57)	0.700 (56)	−1.3 pp	0.688 (55)	0.650 (52)	−3.8 pp
All 513	513	—	0.618 (317)	0.635 (326)	+1.8 pp	0.622 (319)	0.608 (312)	−1.4 pp

Table 15: Official accuracy, APC vs. ForkServe, tp=2 and tp=4. Parentheses are correct counts. The last row is micro-average over the union of the seven files ($5 \times 80 + 73 + 40 = 513$). At two GPUs ForkServe is ahead on six files and on the pool (+9 problems). At four GPUs the pool is within seven problems; no file differs by more than three.

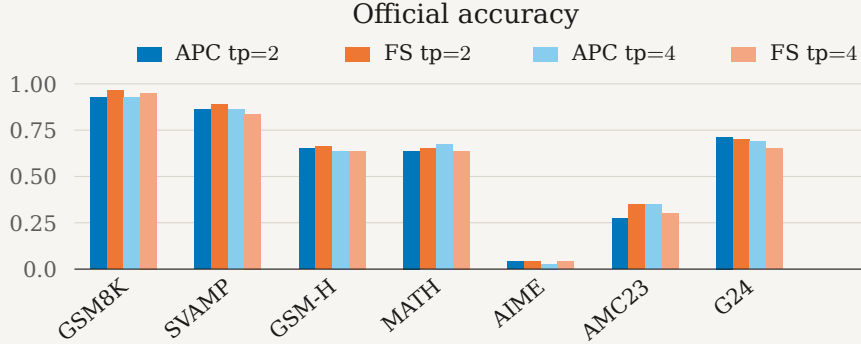


Figure 6: Accuracy, four stacks on each file. Dark bars are tp=2, light bars tp=4. ForkServe (orange) tracks APC (blue) across both widths; AIME is near zero on all four.

Dataset	Peak KV (tokens)			e2e (s), tp=2			e2e (s), tp=4		
	APC	FS	FS/APC	APC	FS	vs. APC	APC	FS	vs. APC
GSM8K	1582	1182	0.75	36.4	35.4	−2.8%	25.1	23.5	−6.1%
SVAMP	1359	959	0.71	35.4	35.7	+1.0%	23.2	23.8	+2.4%
GSM-Hard	1462	1062	0.73	44.1	44.1	−0.1%	29.3	29.2	−0.1%
MATH-500	1765	1349	0.76	66.5	66.2	−0.4%	43.7	43.5	−0.4%
AIME	2118	1702	0.80	67.4	67.3	−0.1%	44.2	44.4	+0.5%
AMC23	1920	1504	0.78	33.7	33.5	−0.5%	22.2	22.0	−0.9%
Game24	1289	793	0.62	44.6	44.3	−0.7%	29.2	29.2	−0.1%

Table 16: Storage and generate time. Peak KV is the same at tp=2 and tp=4. ForkServe is below APC on every file (0.62–0.80×). Wall-clock stays within 3% of APC except GSM8K at four GPUs (−6.1%), where the winner stops early.

Dataset	Fan-out (s)				Decode tokens, tp=2		Decode tokens, tp=4		
	APC t2	FS t2	APC t4	FS t4	APC	FS	APC	FS	FS
GSM8K	0.58	0.95	0.45	0.68	40960	21530	40960	21369	
SVAMP	0.57	0.86	0.46	0.63	40960	16661	40960	16950	
GSM-Hard	0.58	0.85	0.44	0.64	40960	30015	40960	29800	
MATH-500	0.58	0.94	0.47	0.68	61440	44006	61440	45085	
AIME	0.56	1.08	0.44	0.73	56064	55790	56064	55610	
AMC23	0.30	0.55	0.24	0.38	30720	29307	30720	29304	
Game24	0.70	0.84	0.50	0.63	40960	36315	40960	35253	

Table 17: Fan-out work and winner decode tokens. ForkServe pays abort and copy-on-write, so fan-out is 1.2–1.9× APC; that step is still < 4% of T_{e2e} . APC always emits the decode budget; ForkServe stops early on GSM8K and SVAMP, which is why those files can be slightly faster at matched W .

Dataset	FS e2e ₂ (s)	FS e2e ₄ (s)	4/2	APC e2e 4/2	peak FS/APC	dec. share
GSM8K	35.4	23.5	0.66	0.69	0.75	0.97
SVAMP	35.7	23.8	0.67	0.66	0.71	0.97
GSM-Hard	44.1	29.2	0.66	0.66	0.73	0.98
MATH-500	66.2	43.5	0.66	0.66	0.76	0.99
AIME	67.3	44.4	0.66	0.66	0.80	0.98
AMC23	33.5	22.0	0.66	0.66	0.78	0.98
Game24	44.3	29.2	0.66	0.65	0.62	0.98

Table 18: Tensor-parallel scaling and decode share of e2e (ForkServe, tp=2). Four ranks cut wall-clock by 0.66 on every file, matching APC’s own 4/2 ratio. Peak is unchanged. Decode dominates, so the fan-out tax does not set T_{e2e} .

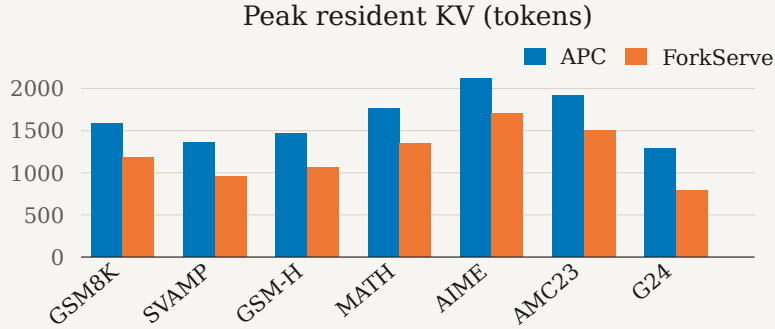


Figure 7: Peak KV, shared by settings E and F. Every file is below APC. Game24 is the largest abort gap (0.62×); AIME the smallest (0.80×), still 20% below the hashed union.

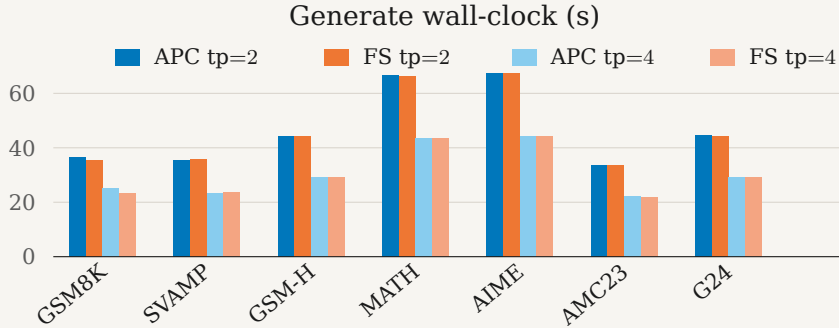


Figure 8: End-to-end generate time. Orange sits on blue in each tensor-width pair. Light bars (tp=4) are 0.66× the dark bars; latency follows decode on more ranks, not a change in M .

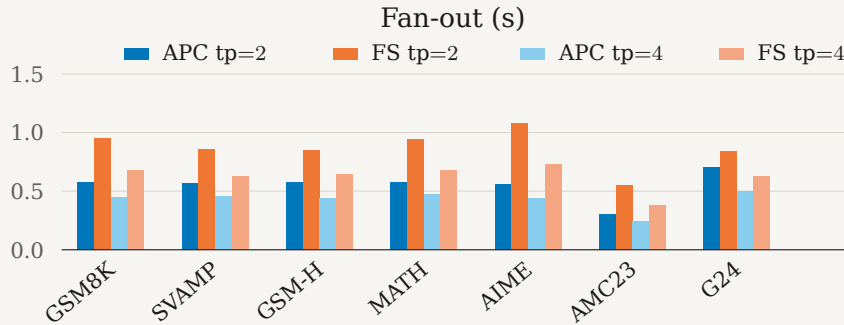


Figure 9: Fan-out (branch prefill plus abort). ForkServe is higher at matched $W=L + k\bar{\ell}$ because of copy-on-write and abort. The gap is tenths of a second against 22–67s generate, so it does not move the accuracy or e2e comparison.